UD5. Programación de bases de datos

Bloque 2. Variables e funcións do sistema

Neste bloque estudarase como utilizar variables e funcións incorporadas de MariaDB para construír consultas SQL máis flexibles e adaptar o comportamento do servidor ás necesidades de cada aplicación.

1. Variables do sistema en MariaDB

1.1. Que son as variables do sistema?

MariaDB utiliza variables do sistema para almacenar valores que determinan o comportamento e a configuración do servidor. Estas variables permiten consultar características do sistema e, nalgúns casos, modificar a súa configuración.

Entre outros aspectos, as variables do sistema permiten controlar:

- O número máximo de conexións simultáneas.
- O tempo máximo de espera dunha conexión inactiva.
- O porto no que escoita o servidor.
- O motor de almacenamento predeterminado.
- A configuración de caracteres e comparación de textos.

Exemplo de variables do sistema

A variable `max_connections` indica o número máximo de conexións simultáneas que pode aceptar o servidor.

```
SHOW VARIABLES LIKE 'max_connections';
```

Tamén se pode consultar o tempo de espera dunha conexión interactiva:

```
SHOW VARIABLES LIKE 'interactive_timeout';
```

1.2. Consultar variables do sistema

A instrución `SHOW VARIABLES` permite consultar as variables de configuración de MariaDB.

```
-- Consultar todas as variables
SHOW VARIABLES;

-- Consultar variables que coincidan cun patrón
SHOW VARIABLES LIKE 'character_set%';

-- Consultar unha variable concreta
SHOW VARIABLES LIKE 'max_connections';
```

O operador `LIKE` permite filtrar os resultados utilizando patróns. O carácter `%` representa calquera secuencia de caracteres.

1.3. Modificar variables do sistema

Algunhas variables poden modificarse durante a execución do servidor mediante a instrución `SET`. O alcance da modificación depende de se se utiliza `GLOBAL` ou `SESSION`.

> **Variables GLOBAL e SESSION**
>
> - **GLOBAL:** establece o valor global dunha variable para o servidor. Afecta ás novas conexións, agás que a variable teña un comportamento específico diferente.
> - **SESSION:** modifica o valor para a conexión actual. Non altera o valor global nin o doutras conexións.

> **Exemplo: modificar unha variable GLOBAL**
>
> ```
> -- Consultar o valor global
> SHOW GLOBAL VARIABLES LIKE 'max_connections';
>
> -- Modificar o número máximo de conexións
> SET GLOBAL max_connections = 200;
>
> -- Comprobar o novo valor
> SHOW GLOBAL VARIABLES LIKE 'max_connections';
> ```

Exemplo: modificar unha variable SESSION

```
-- Consultar o valor da sesión actual
SHOW SESSION VARIABLES LIKE 'wait_timeout';

-- Modificar o tempo de espera da conexión actual
SET SESSION wait_timeout = 600;

-- Comprobar o cambio
SHOW SESSION VARIABLES LIKE 'wait_timeout';
```

Non todas as variables permiten modificar os seus valores. Ademais, algunhas modificacións requiren privilexios de administración. Un cambio realizado mediante `SET GLOBAL` normalmente non se conserva despois de reiniciar o servidor, salvo que tamén se modifique a configuración persistente correspondente.

Para coñecer as variables dispoñibles e as súas características pódese consultar a documentación oficial de MariaDB.

2. Variables de usuario

2.1. Concepto

As variables de usuario permiten almacenar valores temporais durante a execución dunha conexión. A diferenza das variables do sistema, non configuran o servidor: utilízanse para gardar datos intermedios e reutilizalos en diferentes instrucións SQL.

As variables de usuario:

- Comezan co carácter `@`.
- Non necesitan unha declaración previa.
- Créanse cando se lles asigna un valor.
- Están dispoñibles na conexión actual.
- Desaparecen ao finalizar a conexión.

2.2. Asignación de valores

A instrución `SET` permite asignar valores ás variables de usuario.

```
-- Asignar un valor numérico
SET @idade = 25;

-- Asignar unha cadea de texto
SET @nome = 'Ana';

-- Asignar unha data
SET @data_actual = CURDATE();

-- Consultar o contido das variables
SELECT @idade, @nome, @data_actual;
```

Tamén se pode utilizar o operador de asignación `:=` en determinados contextos SQL.

```
SET @contador := 10;

SELECT @contador;
```

2.3. Utilización en consultas SQL

As variables de usuario poden empregarse en expresións, condicións, operacións matemáticas e consultas.

Exemplo: utilizar unha variable nunha condición

```
-- Establecer o curso que queremos consultar
SET @curso = 'DAW1';

-- Consultar os alumnos dese curso
SELECT *
FROM alumnos
WHERE curso = @curso;
```

Exemplo: realizar operacións matemáticas

```
SET @prezo = 100;
SET @desconto = 15;

SELECT
    @prezo AS prezo_inicial,
    @desconto AS porcentaxe_desconto,
    @prezo - (@prezo * @desconto / 100) AS prezo_final;
```

Exemplo: almacenar o resultado dunha consulta

```
-- Gardar o número de alumnos
SET @total_alumnos = (
    SELECT COUNT(*)
    FROM alumnos
);

-- Mostrar o resultado
SELECT @total_alumnos AS total_alumnos;
```

Importante: variables de usuario e variables locais
As variables de usuario, como `@total` , pertencen á conexión e poden utilizarse en distintas instrucións SQL. As variables locais, que se estudarán máis adiante, decláranse con `DECLARE` dentro de procedementos ou bloques de programación e só existen no ámbito correspondente.

3. Funcións incorporadas de MariaDB

3.1. Que son as funcións incorporadas?

MariaDB proporciona un conxunto de funcións que permiten realizar operacións habituais directamente nas consultas SQL. Estas funcións reciben un ou varios argumentos e devolven un resultado.

Poden utilizarse en instrucións `SELECT` , condicións `WHERE` , expresións, agrupacións e ordenacións.

```
SELECT UPPER('mariadb');
SELECT ROUND(15.678, 2);
SELECT CURDATE();
```

3.2. Funcións de cadeas de caracteres

Permiten manipular textos, obter partes dunha cadea, cambiar maiúsculas e minúsculas ou coñecer a súa lonxitude.

Función	Descrición	Exemplo
`UPPER()`	Converte o texto a maiúsculas.	`UPPER('ana')` → ANA
`LOWER()`	Converte o texto a minúsculas.	`LOWER('ANA')` → ana
`LENGTH()`	Devolve a lonxitude en bytes.	`LENGTH('SQL')` → 3
`CHAR_LENGTH()`	Devolve o número de caracteres.	`CHAR_LENGTH('María')` → 5
`CONCAT()`	Une varias cadeas.	`CONCAT('Ana', ' López')`
`SUBSTRING()`	Extrae unha parte dunha cadea.	`SUBSTRING('MariaDB', 1, 5)`
`TRIM()`	Elimina espazos ao principio e ao final.	`TRIM(' texto ')`
`REPLACE()`	Substitúe unha parte do texto.	`REPLACE('ola mundo', 'mundo', 'SQL')`

Exemplo: transformar datos de texto

```
SELECT
    nome,
    UPPER(nome) AS nome_maiusculas,
    LOWER(nome) AS nome_minusculas,
    CHAR_LENGTH(nome) AS lonxitude
FROM alumnos;
```

3.3. Funcións de datas e horas

MariaDB incorpora funcións para obter a data e hora actuais, extraer partes dunha data e realizar operacións temporais.

Función	Descrición	Exemplo
`CURDATE()`	Devolve a data actual.	`CURDATE()`
`NOW()`	Devolve a data e hora actuais.	`NOW()`
`CURTIME()`	Devolve a hora actual.	`CURTIME()`
`YEAR()`	Extrae o ano dunha data.	`YEAR('2026-05-20')`
`MONTH()`	Extrae o mes dunha data.	`MONTH('2026-05-20')`
`DAY()`	Extrae o día do mes.	`DAY('2026-05-20')`
`DATEDIFF()`	Calcula a diferenza en días entre dúas datas.	`DATEDIFF('2026-06-01', '2026-05-20')`
`DATE_ADD()`	Engade un intervalo a unha data.	`DATE_ADD(CURDATE(), INTERVAL 7 DAY)`
`DATE_FORMAT()`	Formata unha data.	`DATE_FORMAT(NOW(), '%d/%m/%Y')`

Exemplo: consultar datas e calcular intervalos

```
SELECT
    CURDATE() AS data_actual,
    NOW() AS data_hora,
    YEAR(CURDATE()) AS ano,
    DATE_ADD(CURDATE(), INTERVAL 30 DAY) AS data_futura;
```

3.4. Funcións numéricas

As funcións numéricas permiten realizar cálculos e operacións matemáticas sobre valores.

Función	Descrición	Exemplo
`ROUND()`	Redondea un número.	`ROUND(15.678, 2)` → 15.68
`CEIL()`	Redondea cara arriba.	`CEIL(15.2)` → 16
`FLOOR()`	Redondea cara abaixo.	`FLOOR(15.8)` → 15
`ABS()`	Devolve o valor absoluto.	`ABS(-25)` → 25
`MOD()`	Devolve o resto dunha división.	`MOD(10, 3)` → 1
`POWER()`	Calcula unha potencia.	`POWER(2, 3)` → 8
`SQRT()`	Calcula a raíz cadrada.	`SQRT(81)` → 9

Exemplo: calcular prezos con desconto

```
SELECT
    produto,
    prezo,
    ROUND(prezo * 0.90, 2) AS prezo_desconto
FROM produtos;
```

A consulta calcula un desconto do 10 % e redondea o resultado a dous decimais.

3.5. Funcións de conversión

As funcións de conversión permiten transformar valores dun tipo de dato noutro. Son especialmente útiles cando se traballa con datos que deben interpretarse ou presentarse cun formato determinado.

Función	Descrición	Exemplo
`CAST()`	Converte unha expresión a un tipo determinado.	`CAST('123' AS UNSIGNED)`
`CONVERT()`	Realiza unha conversión de tipo.	`CONVERT('2026-05-20', DATE)`

Exemplo: conversión de tipos

```
SELECT
    CAST('150' AS UNSIGNED) AS valor_numerico,
    CAST('2026-05-20' AS DATE) AS data_convertida,
    CAST(25.75 AS CHAR) AS valor_texto;
```

3.6. Funcións condicionais

As funcións condicionais permiten devolver resultados diferentes en función de determinadas condicións. Son útiles para clasificar datos, substituír valores nulos ou crear expresións condicionais dentro dunha consulta.

Función	Descrición
`IF()`	Devolve un valor se a condición é verdadeira e outro se é falsa.
`IFNULL()`	Substitúe un valor NULL por outro valor.
`NULLIF()`	Devolve NULL se dous valores son iguais.
`COALESCE()`	Devolve o primeiro argumento que non sexa NULL.
`CASE`	Permite avaliar varias condicións e devolver un resultado.

Exemplo: función IF()

```
SELECT
    nome,
    nota,
    IF(nota >= 5, 'Aprobado', 'Suspenso') AS resultado
FROM alumnos;
```

Exemplo: función IFNULL()

```
SELECT
    nome,
    IFNULL(telefono, 'Non dispoñible') AS telefono
FROM alumnos;
```

Exemplo: función COALESCE()

```
SELECT
    nome,
    COALESCE(telefono, email, 'Sen contacto') AS contacto
FROM alumnos;
```

A función devolve o primeiro valor que non sexa NULL: o teléfono, o correo electrónico ou o texto alternativo.

Exemplo: expresión CASE

```
SELECT
    nome,
    nota,
    CASE
        WHEN nota >= 9 THEN 'Sobresaliente'
        WHEN nota >= 7 THEN 'Notable'
        WHEN nota >= 5 THEN 'Aprobado'
        ELSE 'Suspenso'
    END AS cualificacion
FROM alumnos;
```

3.7. Funcións de información

MariaDB incorpora funcións que permiten obter información sobre a conexión actual, o usuario, a base de datos seleccionada ou o estado da última operación executada.

Función	Descrición
`DATABASE()`	Devolve o nome da base de datos seleccionada.
`USER()`	Devolve o usuario e o host utilizados para establecer a conexión.
`CURRENT_USER()`	Devolve a conta de MariaDB que se utiliza para comprobar os privilexios.
`VERSION()`	Devolve a versión do servidor MariaDB.
`CONNECTION_ID()`	Devolve o identificador da conexión actual.
`ROW_COUNT()`	Devolve o número de filas afectadas pola instrución anterior.
`LAST_INSERT_ID()`	Devolve o último valor AUTO_INCREMENT xerado na conexión.

Exemplo: consultar información do servidor

```
SELECT
    DATABASE() AS base_datos,
    USER() AS usuario_conexion,
    CURRENT_USER() AS usuario_privilexios,
    VERSION() AS version_servidor,
    CONNECTION_ID() AS id_conexion;
```

Exemplo: obter o último identificador xerado

```
INSERT INTO alumnos (nome)
VALUES ('María');

SELECT LAST_INSERT_ID() AS ultimo_id;
```

Se a columna `id` está definida como `AUTO_INCREMENT`, a función permite recuperar o identificador xerado pola inserción na conexión actual.

4. Utilización combinada de variables e funcións

As variables de usuario e as funcións incorporadas poden combinarse para construír consultas máis flexibles. Unha variable pode almacenar un valor calculado mediante unha función e utilizarse posteriormente noutras instrucións.

Exemplo: consulta de alumnos dunha promoción

```
-- Gardar o ano que queremos consultar
SET @ano = YEAR(CURDATE());

-- Consultar alumnos matriculados nese ano
SELECT
    nome,
    data_matricula,
    YEAR(data_matricula) AS ano_matricula
FROM alumnos
WHERE YEAR(data_matricula) = @ano;
```

Exemplo: clasificación de produtos

```
SET @limite = 50;

SELECT
    nome,
    prezo,
    IF(prezo >= @limite, 'Prezo alto', 'Prezo baixo') AS categoria
FROM produtos
ORDER BY prezo DESC;
```

Este tipo de combinación facilita a reutilización de valores e permite adaptar unha consulta sen ter que modificar todas as expresións nas que se utiliza un mesmo dato.

5. Boas prácticas

- Utilizar `SHOW VARIABLES` para consultar a configuración antes de modificar variables do sistema.

- Diferenciar claramente as variables `GLOBAL`, `SESSION` e as variables de usuario.
- Comprobar os privilexios necesarios antes de modificar variables globais.
- Utilizar nomes de variables de usuario que indiquen claramente o seu contido.
- Escoller a función incorporada máis adecuada para cada operación.
- Ter en conta os valores NULL cando se realizan operacións e comparacións.
- Comprobar os tipos de datos e realizar conversións explícitas cando sexa necesario.
- Consultar a documentación da versión de MariaDB instalada cando se utilicen funcións ou variables específicas.

6. Exercicios prácticos

Os seguintes exercicios permiten practicar a consulta de variables do sistema, o uso de variables de usuario e as funcións incorporadas de MariaDB.

Exercicio 1. Variables do sistema

1. Consulta todas as variables do sistema de MariaDB.
2. Localiza as variables relacionadas co número máximo de conexións.
3. Consulta as variables relacionadas cos xogos de caracteres e as colacións.
4. Consulta o valor global de `max_connections`.
5. Consulta o valor de `wait_timeout` na sesión actual.
6. Modifica unha variable de sesión e comproba o resultado.
7. Compara o valor dunha variable GLOBAL co seu valor SESSION.

Exercicio 2. Variables de usuario

1. Crea unha variable para almacenar o nome dun alumno.
2. Crea variables para almacenar unha nota e o número de faltas.
3. Realiza unha consulta que mostre os valores almacenados.
4. Calcula o resultado dunha operación matemática utilizando variables.
5. Utiliza unha variable nunha condición WHERE.
6. Garda nunha variable o resultado dunha consulta que conte os rexistros dunha táboa.

Exercicio 3. Funcións de cadeas e números

1. Consulta os nomes dos alumnos en maiúsculas e minúsculas.
2. Mostra a lonxitude dos nomes almacenados.

3. Une o nome e os apelidos de cada alumno nunha única columna.
4. Calcula o prezo final dos produtos aplicando un desconto.
5. Redondea os resultados a dous decimais.
6. Utiliza funcións numéricas para calcular o resto dunha división e o valor absoluto dun número.

Exercicio 4. Funcións de datas e conversións

1. Consulta a data e hora actuais do servidor.
2. Mostra por separado o ano, mes e día da data actual.
3. Calcula cantos días transcorreron entre dúas datas.
4. Obtén unha data futura engadindo un intervalo de días.
5. Mostra unha data utilizando o formato día/mes/ano.
6. Realiza conversións entre cadeas de texto, números e datas.

Exercicio 5. Funcións condicionais e de información

1. Clasifica os alumnos como aprobados ou suspensos mediante `IF()`.
2. Substitúe os valores NULL dunha columna por un texto alternativo.
3. Utiliza `COALESCE()` para seleccionar o primeiro dato de contacto dispoñible.
4. Clasifica as notas en diferentes categorías mediante `CASE`.
5. Consulta o nome da base de datos seleccionada e a versión do servidor.
6. Consulta o usuario da conexión e o usuario empregado para comprobar os privilexios.
7. Realiza unha inserción nunha táboa con identificador AUTO_INCREMENT e consulta o último identificador xerado.

Exercicio 6. Consulta integradora

Elabora unha consulta que combine variables de usuario e funcións incorporadas para analizar os datos dunha táboa da base de datos utilizada nas prácticas.

A consulta deberá:

- Utilizar polo menos dúas variables de usuario.
- Incorporar funcións de cadeas e funcións numéricas.
- Utilizar polo menos unha función de datas.
- Aplicar unha función condicional.
- Mostrar información da conexión ou do servidor.

7. Resumo do bloque

Elemento	Utilidade	Exemplos
Variables do sistema	Consultar e modificar a configuración do servidor.	`SHOW VARIABLES`, `SET GLOBAL`
Variables de sesión	Modificar valores para a conexión actual.	`SET SESSION`
Variables de usuario	Almacenar e reutilizar valores nunha conexión.	`@nome`, `SET @total = 10`
Funcións de cadeas	Manipular textos.	`UPPER()`, `CONCAT()`
Funcións de datas	Consultar e operar con datas e horas.	`NOW()`, `DATE_ADD()`
Funcións numéricas	Realizar cálculos matemáticos.	`ROUND()`, `ABS()`
Funcións de conversión	Transformar valores entre tipos de datos.	`CAST()`, `CONVERT()`
Funcións condicionais	Devolver resultados segundo condicións.	`IF()`, `COALESCE()`, `CASE`
Funcións de información	Obter datos do servidor e da conexión.	`DATABASE()`, `VERSION()`

> **Obxectivo alcanzado**
> Ao rematar este bloque, o alumnado debe ser capaz de consultar e modificar variables do sistema cando sexa necesario, utilizar variables de usuario para almacenar valores temporais e incorporar funcións de MariaDB nas consultas para realizar operacións con textos, datas, números e condicións.